

Целые числа

Все предыдущие программы, которые мы писали, работали с текстовыми данными. Действительно, команда `input()` считывает строку текста. Однако во многих случаях нам нужно работать именно с числами. Чтобы в Python создать переменную целого типа данных, нужно опустить кавычки при объявлении переменной. Рассмотрим следующий код:

```
num1 = 7          # num1 - это число
num2 = 10         # num2 - это число
num3 = num1 + num2 # num3 - это число
print(num3)
```

В результате выполнения такой программы будет выведено число 17.

Запомни: числа обозначаются без кавычек, а строки — с кавычками.

Основные операции с числами

В языке Python, как и в математике, над числами можно совершать 4 основные операции (+, −, *, /).

Оператор	Описание
+	сложение
-	вычитание
*	умножение
/	деление

Рассмотрим следующую программу:

```
a = 3
b = 2
print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

Результатом выполнения такой программы будут числа:

5

1

6

1.5

Порядок выполнения операций

В математике существует порядок выполнения операций, определяющий, какие операции должны выполняться раньше других, даже если в выражении они написаны правее. Порядок выполнения операций в Python аналогичен порядку выполнения операций, которые вы изучали на уроках математики.

Рассмотрим следующий код:

```
num1 = 2 + 3 * 4
```

```
num2 = (2 + 3) * 4
```

```
print(num1)
```

```
print(num2)
```

Результатом выполнения такой программы будут числа 14 и 20. В переменной `num1` будет храниться число 14, поскольку первым делом выполняется умножение, а уже затем сложение. В переменной `num2` будет храниться число 20, поскольку скобки изменили приоритет выполнения операций.

Запомни: первым делом выполняется умножение или деление, затем сложение и вычитание. Для изменения порядка выполнения операций понадобятся скобки.

Преобразование типов

Преобразование строки к целому числу. Для того, чтобы преобразовать строку к целому числу, мы используем команду `int()`. Рассмотрим следующий код:

```
s = '1992'
```

```
year = int(s)
```

Переменная `s` имеет строковый тип данных. С помощью команды `int()`, мы преобразовали строку к целому числу и записали результат в переменную с именем `year`. **Мы используем новую команду (точнее, функцию) – `int()`.** Она означает следующее: «Возьми то, что указано в качестве аргумента в скобках, и преврати это в целое число». Таким образом, переменная `year` имеет целый тип данных, а переменная `s` – строковый тип данных.

Напишем программу, которая считывает два целых числа и выводит на экран их сумму. Следующий код тут не сработает:

```
num1 = input()

num2 = input()

print(num1 + num2)
```

Python считает, что в переменных `num1` и `num2` находится **текст**, поскольку команда `input()` по умолчанию считывает именно **текст**. Для того, чтобы явно указать, что требуется работать с переменными целого типа, надо написать так:

```
num1 = int(input())

num2 = int(input())

print(num1 + num2)
```

Запомни: для того, чтобы считать одно целое число, мы пишем следующий код:

```
num = int(input())
```

Преобразование целого числа к строке. Для того, чтобы преобразовать целое число в строку, мы используем команду `str()`. Рассмотрим следующий код:

```
num = 17

s = str(17)
```

Переменная `num` имеет целый тип данных. С помощью команды `str()` мы преобразовали целое число в строку и записали результат в переменную `s`. Таким образом, переменная `num` имеет целочисленный тип данных, а переменная `s` – строковый тип данных.

Примечания

Примечание 1. При работе с целочисленными переменными часто нужно осуществлять присвоение значению переменной целой математической формулы:

```
<имя переменной> = <математическая формула>
```

имя переменной

=

математическая формула

Примечание 2. Сокращение `int` происходит от английского **integer** – целый.

Примечание 3. В большинстве языков программирования переменная целого типа имеет ограничения. Например, в языке C# переменная целого типа может принимать значения из диапазона $[-263; 263-1]$ $[-2^{63}; 2^{63}-1]$. В Python

реализована так называемая **длинная арифметика**, то есть, по сути, переменная целого типа не имеет ограничений.

Примечание 4. Минус может использоваться для обозначения отрицательных чисел. А вот операции $+$, $/$ и $*$ всегда стоят между двумя переменными и что-то с ними делают.

```
num1 = -6      # унарный минус
num2 = 17 - 7   # бинарный минус
```

Дополнительные операции

Мы познакомились с 44 основными математическими операциями в языке Python: сложение $(+)(+)$, вычитание $(-)(-)$, умножение $(*)(*)$ и деление $(/)(/)$. Добавив ещё 33 операции, мы получим инструментарий, достаточный для написания 99%99% программ.

Оператор	Описание
<code>**</code>	Возведение в степень
<code>%</code>	Остаток от деления
<code>//</code>	Целочисленное деление

Возведение в степень

Оператор возведения в степень `a ** n` возводит число a в степень n . Рассмотрим работу данного оператора на примерах:

```
print(2 ** 0)
print(2 ** 1)
print(2 ** 2)
print(2 ** 3)
print(2 ** (-1))
```

Результатом выполнения такой программы будет:

```
1
2
4
8
```

0.5

Обратите внимание: оператор возведения в степень `**` может возводить не только в положительную степень, но и в отрицательную. Аналогично, основание степени `a` также может быть числом отрицательным.

Запомни: в отличие от многих языков программирования, в языке Python для возведения в степень используется оператор `**`, а не `^`.

Обратите внимание: оператор возведения в степень (`**`) является правоассоциативным (значение выражения вычисляется справа налево) в соответствии с правилами математики. Таким образом, выражение `x ** y ** z` вычисляется как `x ** (y ** z)`.

Результатом работы следующей программы:

```
print(2 ** 2 ** 3)      # 2 ** (2 ** 3) = 2 ** 8
```

будет:

256

Целочисленное деление

Для положительных чисел оператор целочисленного деления ведёт себя как обычное деление, за исключением того, что он отбрасывает десятичную часть результата. Рассмотрим работу данного оператора на примерах:

```
print(10 // 3)
```

```
print(10 // 4)
```

```
print(10 // 5)
```

```
print(10 // 6)
```

```
print(10 // 12)
```

Результатом выполнения такой программы будет:

3

2

2

1

0

При делении отрицательных чисел необходимо помнить, что результат целочисленного деления не превосходит частное. Другими словами, округление берётся в меньшую сторону (число $-4-4$ меньше, чем число $-3-3$).

Результатом работы следующей программы:

```
print(10 // 3)

print(-10 // 3)
```

будут числа:

```
3    # округление в меньшую сторону
-4   # округление в меньшую сторону
```

Деление с остатком

Оператор деления с остатком возвращает остаток от деления двух целых чисел. Рассмотрим работу данного оператора на примерах:

```
print(10 % 3)

print(10 % 4)

print(10 % 5)

print(10 % 6)

print(10 % 12)

print(10 % 20)
```

Результатом выполнения такой программы будет:

```
1
2
0
4
10
10
```

Запомни: при нахождении остатка от деления на целое число n может получаться результат $0, 1, 2, \dots, n-1$. Например, при делении на 22 возможны остатки 0, 10, 1, при делении на 33 возможны остатки 0, 1, 20, 1, 2 и т.д.

Примечания

Примечание 1. Оператор нахождения остатка очень полезен при решении многих задач. Например, число делится на n нацело тогда и только тогда, когда остаток от деления на n равен 00.

Примечание 2. Операторы `//` и `%` имеют такой же приоритет, как и операторы умножения и обычного деления.

Примечание 3. Наивысший приоритет имеет оператор возведения в степень `**`.

Примечание 4. Полезно прочитать про [модулярную арифметику](#) в математике.

Примечание 5. Обратите внимание: результатом деления `n % m` при `n < m` является число `n`. Например, `5 % 9 = 5`, `3 % 13 = 3` и т.д.